

# Module 4

Smart Contract Lifecycle (Foundry)

# What is Foundry?

## A blazing fast smart contract development toolkit

Written in Rust (not JavaScript)

Ethereum-native workflow

Perfect for serious developers

## Core Tools:

Forge: Compile, test, build, deploy

Cast: CLI for contract interaction

Anvil: Local Ethereum node

Chisel: Solidity REPL

## Why Foundry?

Faster than Hardhat

Better for testing and auditing

Closer to the EVM

# Installation

```
# Install foundryup (installer)
curl -L https://foundry.paradigm.xyz
    | bash

# Restart terminal
source ~/.bashrc # or ~/.zshrc

# Verify installation
forge --version
cast --version
```

## One-line install

Installs all 4 tools

Works on Mac, Linux,  
Windows (WSL)

Automatic updates via  
foundryup



# Writing Smart Contracts

```
pragma solidity ^0.8.20;

contract SimpleVault {
    mapping(address user => uint256 balance) public balances;

    event Deposit( address indexed user, uint256 amount);
    event Withdrawal(address indexed user, uint256 amount);

    function deposit()
        external payable {
            require(msg.value > 0, "Cannot deposit zero");
            balances[msg.sender]
                += msg.value;
            emit Deposit(msg.sender,
                        msg.value);
        }

    function withdraw(uint256 _amount)
        external {
            require(_amount > 0, "Cannot deposit zero");
            require(balances[msg.sender] >= _amount,
                "InsufficientBalance");
            balances[msg.sender] -= _amount;
            (Bool success,) =
                msg.sender.call{value: _amount}("");
            emit Withdrawal(msg.sender,
                        _amount);
        }
}
```

## Store in src/

SimpleVault.sol

Include events for  
off-chain listeners

Emit events at key  
state changes

## Basic File Structure

| ▼ DEMO |                 |   |
|--------|-----------------|---|
| >      | .github         | ● |
| >      | lib             |   |
| >      | script          | ● |
| ▼      | src             | ● |
|        | SimpleVault.sol | U |
| >      | test            | ● |
| ◆      | .gitignore      | U |
| ◆      | .gitmodules     | A |
| ≡      | foundry.lock    | U |
| ⚙      | foundry.toml    | U |
| ℹ      | README.md       | U |

# Compiling: forge build

## Compile your contract:

\$ forge build

## What happens:

- Reads Solidity source files
- Compiles to bytecode
- Outputs artifacts to /out
- Uses compiler in foundry.toml

## Configuration (foundry.toml):

```
solc = "0.8.20"
optimiser = true
optimiser_runs = 200
```

## Why Optimiser Matters:

- Reduces bytecode size
- Lowers gas costs
- Changes bytecode hash (verification!)

## Successful Compilation

```
SimpleVault.sol U X
src > SimpleVault.sol > ...
1 // SPDX-License-Identifier: UNLICENSED
2 pragma solidity 0.8.25;
3
4 contract SimpleVault {
5     mapping(address user => uint256 balance) public balances;
6
7     event Deposit(address indexed user, uint256 amount);
8     event Withdrawal(address indexed user, uint256 amount);
9
10    function deposit() external payable {
11        require(msg.value > 0, "Cannot deposit zero");
12        balances[msg.sender] += msg.value;
13        emit Deposit(msg.sender, msg.value);
14    }
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
home@Myless-MBP demo % forge build
[.] Compiling...
[.] Compiling 3 files with Solc 0.8.25
[.] Solc 0.8.25 finished in 50.69ms
Compiler run successful!
home@Myless-MBP demo %
```

# Example .toml file

foundry.toml M X

foundry.toml

```
1  [profile.default]
2  src = "src"
3  out = "out"
4  libs = ["lib"]
5
6  solc = "0.8.20"
7  optimizer=true
8  runs=200
9
10 # See more config options https://github.com/foundry-rs/foundry/blob/master/crates/config/README.md#all-options
11
12 [rpc_endpoints]
13 sepolia = "${SEPOLIA_RPC_URL}"
14
15 [etherscan]
16 sepolia = {key = "${ETHERSCAN_API_KEY}", chain = 11155111}
17
```



# Testing: forge test

```
pragma solidity ^0.8.20;
```

```
import "forge-std/Test.sol";
import "../src/SimpleVault.sol";
```

```
contract SimpleVaultTest is Test {
    SimpleVault vault;
    address alice;
    address bob;
    address owner
```

```
    function setUp() public {
        Alice = vm.addr(1);
        Bob = vm.addr(2);
        Owner = makeAddr("Owner");
```

```
        deal(Alice, 1 ether);
        deal(Bob, 1 ether);
```

```
        vm.prank(owner);
        vault = new SimpleVault();
```

```
}
```

```
function testDeposit() public {
    vm.prank(Alice);
    vault.deposit{value: 1 ether}();
    assertEq(vault.balances(Alice), 1 ether);
```

```
}
```

```
}
```

Create in test/

SimpleVault.t.sol

|   |                   |   |
|---|-------------------|---|
| > | script            | ● |
| > | src               | ● |
| ∨ | test              | ● |
|   | SimpleVault.t.sol | U |
| ◆ | .gitignore        | U |

setUp():

- Runs before each test
- Uses Foundry vm cheat codes to create users and supply them with ETH

testDeposit() tests

the deposit function by asserting the callers balance has increased



# Running Tests: forge test

## Execute all tests:

\$ forge test

## What forge does:

Spins up local EVM

Runs setUp() before each test

Executes testXXX() functions

Reports gas usage per function

Shows failures/successes

## Advanced Testing:

Fuzz testing: random inputs

Invariant testing: property-based

Fork testing: test against Mainnet

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● home@Myless-MBP demo % forge test
[·] Compiling...
[·] Compiling 20 files with Solc 0.8.25
[·] Solc 0.8.25 finished in 514.11ms
Compiler run successful!

Ran 1 test for test/SimpleVault.t.sol:SimpleVaultTest
[PASS] testDeposit() (gas: 43097)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 4.85ms (545.88µs CPU time)

Ran 1 test suite in 129.82ms (4.85ms CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)
○ home@Myless-MBP demo %
```



# Deployment Scripts

```
pragma solidity ^0.8.20;

import "forge-std/Script.sol";
import "../src/SimpleVault.sol";

contract DeployVault is Script {
    function run() external {
        vm.startBroadcast();
        new SimpleVault();
        vm.stopBroadcast();
    }
}
```

Store in script/

DeployVault.s.sol

vm.startBroadcast()

starts sending txs

Deterministic & replayable  
deployment logic

# Deployment Paths: Local vs Testnet

## Local Development (Anvil)

1. Start Anvil

\$ anvil

2. Deploy Contract

\$ forge script  
--rpc-url  
localhost:8545

3. Test Locally

\$ cast call/send

## Testnet (Sepolia)

1. Import Key Securely

\$ cast wallet import  
--interactive

2. Deploy

\$ forge script  
--rpc-url \$RPC\_URL  
--account my-account  
--broadcast

3. Verify Contract

\$ forge verify-contract

# Interacting: cast call/send

```
# Read balance
cast call CONTRACT_ADDRESS \
  "balances(address)" YOUR_ADDRESS \
  --rpc-url $RPC_URL

# Send a deposit
cast send CONTRACT_ADDRESS \
  "deposit()" \
  --value 1ether \
  --account my-account \
  --rpc-url $RPC_URL
```

**call = read-only**

**send = state-changing**

No frontend needed

Perfect for backends

Use secure imported key  
with --account flag



# Verification: forge verify-contract

## Verify contract source on Etherscan:

```
$ forge verify-contract \  
  CONTRACT_ADDRESS \  
  src/SimpleVault.sol:SimpleVault \  
  --chain-id 11155111 \  
  --etherscan-api-key $ETHERSCAN_API_KEY
```

## What Happens:

- Forge recompiles contract
- Matches resulting bytecode
- Submits source to Etherscan

**If bytecode matches → Source is verified**

## Why Verification Matters:

- Users can read your code
- Builds trust
- Required for audits

# Complete Foundry Workflow

Write

`src/`

Compile

`forge build`

Test

`forge test`

Deploy

`forge script`

Interact

`cast call/send`

Verify

`forge verify`

# Secure Key Management: cast wallet

```
# Import a private key securely  
cast wallet import --interactive
```

```
Enter private key: [hidden input]  
Set password: [hidden input]  
Key imported as 'my-account'
```

```
# Use in deployment script  
vm.startBroadcast(  
  vm.envAddress("MY_ACCOUNT")  
);
```

```
# Or reference by name  
--account my-account
```

## Encrypted key storage

In ~/.foundry/keystores/

Never in plain text

Password-protected access

Safe for CI/CD with secrets



# Foundry Best Practices

## Testing First

Write tests before deployment

Aim for high coverage

Test edge cases and failures

## Secure Key Management

NEVER store private keys in plain text

Use: `cast wallet import --interactive`

Keys stored encrypted in `~/.foundry/`

Reference by account name in scripts

## Script Organisation

Keep scripts in `script/`

Make deployments idempotent

Log deployment addresses



# Testing & Security Checklist

## Before Deployment

- Write comprehensive unit tests
- Test all edge cases and failures
- Aim for 80%+ code coverage
- Run fuzzing tests

## Security Review

- Check for reentrancy issues
- Verify access control
- Audit event emissions

## Deployment Checklist

- Use cast wallet import for keys
- Verify contract source on chain
- Test on testnet first
- Log all deployment addresses

# Module 4 Recap: Smart Contract Lifecycle

- ✓ Foundry is the modern Ethereum development toolkit
- ✓ Write contracts in `src/`
- ✓ Compile with `forge build`
- ✓ Test with `forge test` (before deployment!)
- ✓ Deploy with `forge script`
- ✓ Interact with `cast call/send`
- ✓ Verify on Etherscan

## Key Principles:

- Test thoroughly before mainnet
- NEVER store keys in plain text
- Use `cast wallet import` for security
- Verify all deployments

## Next Steps:

- Set up Foundry locally
- Write comprehensive tests
- Deploy your first contract